

1. 8 PUZZLE PROBLEM

Code:

```
# Name: Desavath Srujana Bai  
# Register Number: 192571067
```

```
print("Name : Desavath Srujana Bai")  
print("Register Number : 192571067")
```

```
goal = [1,2,3,4,5,6,7,8,0]
```

```
print("\nEnter the initial state (9 numbers):")  
state = list(map(int, input().split()))
```

```
if state == goal:  
    print("Goal State Reached!")  
else:  
    count = 0  
    for i in range(9):  
        if state[i] != goal[i]:  
            count += 1  
    print("Misplaced Tiles =", count)
```

```
print("\nInitial State:")  
for i in range(0,9,3):  
    print(state[i:i+3])
```

```
print("\nGoal State:")  
for i in range(0,9,3):  
    print(goal[i:i+3])
```

Output:

```
Name : Desavath Srujana Bai  
Register Number : 192571067  
  
Enter the initial state (9 numbers):  
1 2 3 4 5 6 7 0 8  
  
Misplaced Tiles = 2  
  
Initial State:  
[1, 2, 3]  
[4, 5, 6]  
[7, 0, 8]  
  
Goal State:  
[1, 2, 3]  
[4, 5, 6]  
[7, 8, 0]
```

2. 8 QUEEN PROBLEM

Code:

```
# Name: Desavath Srujana Bai
# Register Number: 192571067

print("Name : Desavath Srujana Bai")
print("Register Number : 192571067")

N = int(input("Enter the number of queens: "))
x = [-1] * N

def safe(r, c):
    for i in range(c):
        if x[i] == r or abs(x[i] - r) == abs(i - c):
            return False
    return True

def queen(c):
    if c == N:
        print("\nSolution:")
        for i in range(N):
            print(x[i], end=" ")
        return True

    for r in range(N):
        if safe(r, c):
            x[c] = r
            if queen(c + 1):
                return True
    return False

if not queen(0):
    print("No Solution")
```

Output:

```
Name : Desavath Srujana Bai
Register Number : 192571067
Enter the number of queens: 8

Solution:
0 4 7 5 2 6 1 3
```

3. WATER JUG PROBLEM

Code:

Name: Desavath Srujana Bai

Register Number: 192571067

```
print("Name : Desavath Srujana Bai")
```

```
print("Register Number : 192571067")
```

```
j1 = int(input("Enter capacity of Jug 1: "))
```

```
j2 = int(input("Enter capacity of Jug 2: "))
```

```
target = int(input("Enter target amount: "))
```

```
a = 0
```

```
b = 0
```

```
while b != target:
```

```
    if a == 0:
```

```
        a = j1
```

```
    elif b == j2:
```

```
        b = 0
```

```
    else:
```

```
        t = min(a, j2 - b)
```

```
        a -= t
```

```
        b += t
```

```
print("Jug1 =", a, "Jug2 =", b)
```

```
print("Target Reached!")
```

Output:

```
Name : Desavath Srujana Bai
Register Number : 192571067
Enter capacity of Jug 1: 4
Enter capacity of Jug 2: 3
Enter target amount: 2
```

```
Jug1 = 4 Jug2 = 0
```

```
Jug1 = 1 Jug2 = 3
```

```
Jug1 = 1 Jug2 = 0
```

```
Jug1 = 0 Jug2 = 1
```

```
Jug1 = 4 Jug2 = 1
```

```
Jug1 = 2 Jug2 = 3
```

```
Jug1 = 2 Jug2 = 0
```

```
Target Reached!
```

4. VACUUM CLEANER PROBLEM

Code:

```
# Name: Desavath Srujana Bai
# Register Number: 192571067
```

```
print("Name : Desavath Srujana Bai")
print("Register Number : 192571067")

A = input("Enter Room A status (Dirty/Clean): ")
B = input("Enter Room B status (Dirty/Clean): ")

print("\nInitial State")
print("Room A =", A)
print("Room B =", B)

print("\nVacuum starts at Room A")

if A.lower() == "dirty":
    print("Room A is Dirty")
    print("Cleaning Room A...")
    A = "Clean"
else:
    print("Room A is already Clean")

print("\nMove to Room B")

if B.lower() == "dirty":
    print("Room B is Dirty")
    print("Cleaning Room B...")
    B = "Clean"
else:
    print("Room B is already Clean")

print("\nFinal State")
print("Room A =", A)
print("Room B =", B)

print("\nAll rooms are clean.")
```

Output:

```
Name : Desavath Srujana Bai
Register Number : 192571067

Enter Room A status (Dirty/Clean): Dirty
Enter Room B status (Dirty/Clean): Dirty

Initial State
Room A = Dirty
Room B = Dirty

Vacuum starts at Room A
Room A is Dirty
Cleaning Room A...

Move to Room B
Room B is Dirty
Cleaning Room B...

Final State
Room A = Clean
Room B = Clean

All rooms are clean.
```

5. 5. BREADTH FIRST SEARCH (BFS)

Code:

```
# Name: Desavath Srujana Bai
# Register Number: 192571067

print("Name : Desavath Srujana Bai")
print("Register Number : 192571067")

visited = []
queue = []
graph = {}

n = int(input("Enter number of vertices: "))

for i in range(n):
    vertex = input("Enter vertex: ")
    neighbors = input("Enter adjacent vertices (space separated): ").split()
    graph[vertex] = neighbors

def bfs(start):
    visited.append(start)
    queue.append(start)

    while queue:
        node = queue.pop(0)
        print(node, end=" ")

        for i in graph[node]:
            if i not in visited:
                visited.append(i)
                queue.append(i)

start = input("\nEnter starting vertex: ")

print("BFS Traversal:")
bfs(start)
```

Output:

```
Name : Desavath Srujana Bai
Register Number : 192571067

Enter number of vertices: 4
Enter vertex: A
Enter adjacent vertices (space separated): B C
Enter vertex: B
Enter adjacent vertices (space separated): D
Enter vertex: C
Enter adjacent vertices (space separated): D
Enter vertex: D
Enter adjacent vertices (space separated):

Enter starting vertex: A
BFS Traversal:
A B C D
```

6. 6.DEPTH FIRST SEARCH (DFS)

Code:

```
# Name: Desavath Srujana Bai
# Register Number: 192571067

print("Name : Desavath Srujana Bai")
print("Register Number : 192571067")

visited = []
graph = {}

n = int(input("Enter number of vertices: "))

for i in range(n):
    vertex = input("Enter vertex: ")
    neighbors = input("Enter adjacent vertices (space separated): ").split()
    graph[vertex] = neighbors

def dfs(node):
    if node not in visited:
        print(node, end=" ")
        visited.append(node)

        for i in graph[node]:
            dfs(i)

start = input("\nEnter starting vertex: ")

print("\nDFS Traversal:")
dfs(start)
```

Output:

```
Name : Desavath Srujana Bai
Register Number : 192571067

Enter number of vertices: 4
Enter vertex: A
Enter adjacent vertices (space separated): B C
Enter vertex: B
Enter adjacent vertices (space separated): D
Enter vertex: C
Enter adjacent vertices (space separated): D
Enter vertex: D
Enter adjacent vertices (space separated):

Enter starting vertex: A

DFS Traversal:
A B D C
```

Code:

Name: Desavath Srujana Bai

Register Number: 192571067

```
print("Name : Desavath Srujana Bai")
print("Register Number : 192571067")
```

```
from collections import deque
```

```
q = deque([(3,3,0), []])
moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]
seen = set()
```

```
while q:
    (m,c,b), path = q.popleft()
```

```
    if (m,c,b) in seen:
        continue
```

```
    seen.add((m,c,b))
```

```
    if (m,c,b) == (0,0,1):
        print(path + [(m,c,b)])
        break
```

```
    for x,y in moves:
        nm = m-x if b==0 else m+x
        nc = c-y if b==0 else c+y
```

```
    if 0 <= nm <= 3 and 0 <= nc <= 3:
        if (nm==0 or nm>=nc) and (3-nm==0 or 3-nm>=3-nc):
            q.append(((nm,nc,1-b), path+[(m,c,b)]))
```

Output:

```
Name : Desavath Srujana Bai
Register Number : 192571067
```

```
[(3, 3, 0), (3, 1, 1), (3, 2, 0), (3, 0, 1),
 (3, 1, 0), (1, 1, 1), (2, 2, 0), (0, 2, 1),
 (0, 3, 0), (0, 1, 1), (1, 1, 0), (0, 0, 1)]
```

7. 8. TRAVELLING SALESMAN PROBLEM

Code:

```
# Name: Desavath Srujana Bai
# Register Number: 192571067

print("Name : Desavath Srujana Bai")
print("Register Number : 192571067")

from itertools import permutations

d = [[0,10,15,20],
      [10,0,35,25],
      [15,35,0,30],
      [20,25,30,0]]

cost = 999
path = ()

for p in permutations([1,2,3]):
    c = d[0][p[0]] + d[p[0]][p[1]] + d[p[1]][p[2]] + d[p[2]][0]

    if c < cost:
        cost = c
        path = (0,) + p + (0,)

print("Shortest Path:", path)
print("Minimum Cost:", cost)
```

Output:

```
Name : Desavath Srujana Bai
Register Number : 192571067

Shortest Path: (0, 1, 3, 2, 0)
Minimum Cost: 80
```


8. 9. CRYPT ARITHMETIC PROBLEM

Code:

Name: Desavath Srujana Bai

Register Number: 192571067

```
print("Name : Desavath Srujana Bai")
print("Register Number : 192571067")
```

```
from itertools import permutations
```

```
for p in permutations(range(10), 8):
    S,E,N,D,M,O,R,Y = p
```

```
    if S and M:
```

```
        send = 1000*S + 100*E + 10*N + D
```

```
        more = 1000*M + 100*O + 10*R + E
```

```
        money = 10000*M + 1000*O + 100*N + 10*E + Y
```

```
        if send + more == money:
```

```
            print("SEND =", send)
```

```
            print("MORE =", more)
```

```
            print("MONEY =", money)
```

```
            break
```

Output:

```
Name : Desavath Srujana Bai
```

```
Register Number : 192571067
```

```
SEND = 9567
```

```
MORE = 1085
```

```
MONEY = 10652
```

9. 10. A* ALGORITHM

Python Code:

```
# Name: Desavath Srujana Bai
```

```
# Register Number: 192571067
```

```
print("Name : Desavath Srujana Bai")
```

```
print("Register Number : 192571067")
```

```
graph = {}
```

```
n = int(input("Enter number of nodes: "))
```

```
for i in range(n):
```

```
    node = input("Node: ")
```

```
    graph[node] = {}
```

```
    e = int(input("Edges from " + node + ": "))
```

```
    for j in range(e):
```

```
        v = input("Connected node: ")
```

```
        c = int(input("Cost: "))
```

```
        graph[node][v] = c
```

```
h = {}
```

```
for i in graph:
```

```
    h[i] = int(input("Heuristic of " + i + ": "))
```

```
start = input("Start node: ")
```

```
goal = input("Goal node: ")
```

```
open_list = [start]
```

```
cost = {start: 0}
```

```
parent = {start: None}
```

```
while open_list:
```

```
    x = min(open_list, key=lambda i: cost[i] + h[i])
```

```
    open_list.remove(x)
```

```
    if x == goal:
```

```
        path = []
```

```
        while x:
```

```
            path.append(x)
```

```
            x = parent[x]
```

```
        print("Path:", path[::-1])
```

```
        break
```

```
    for y in graph[x]:
```

```
        g = cost[x] + graph[x][y]
```

```
        if y not in cost or g < cost[y]:
```

```
            cost[y] = g
```

```
            parent[y] = x
```

```
        if y not in open_list:
```

```
            open_list.append(y)
```

Output:

Name : Desavath Srujana Bai
Register Number : 192571067

Enter number of nodes: 3

Node: A

Edges from A: 2

Connected node: B

Cost: 1

Connected node: C

Cost: 4

Node: B

Edges from B: 1

Connected node: C

Cost: 2

Node: C

Edges from C: 0

Heuristic of A: 3

Heuristic of B: 1

Heuristic of C: 0

Start node: A

Goal node: C



Path: ['A', 'B', 'C']